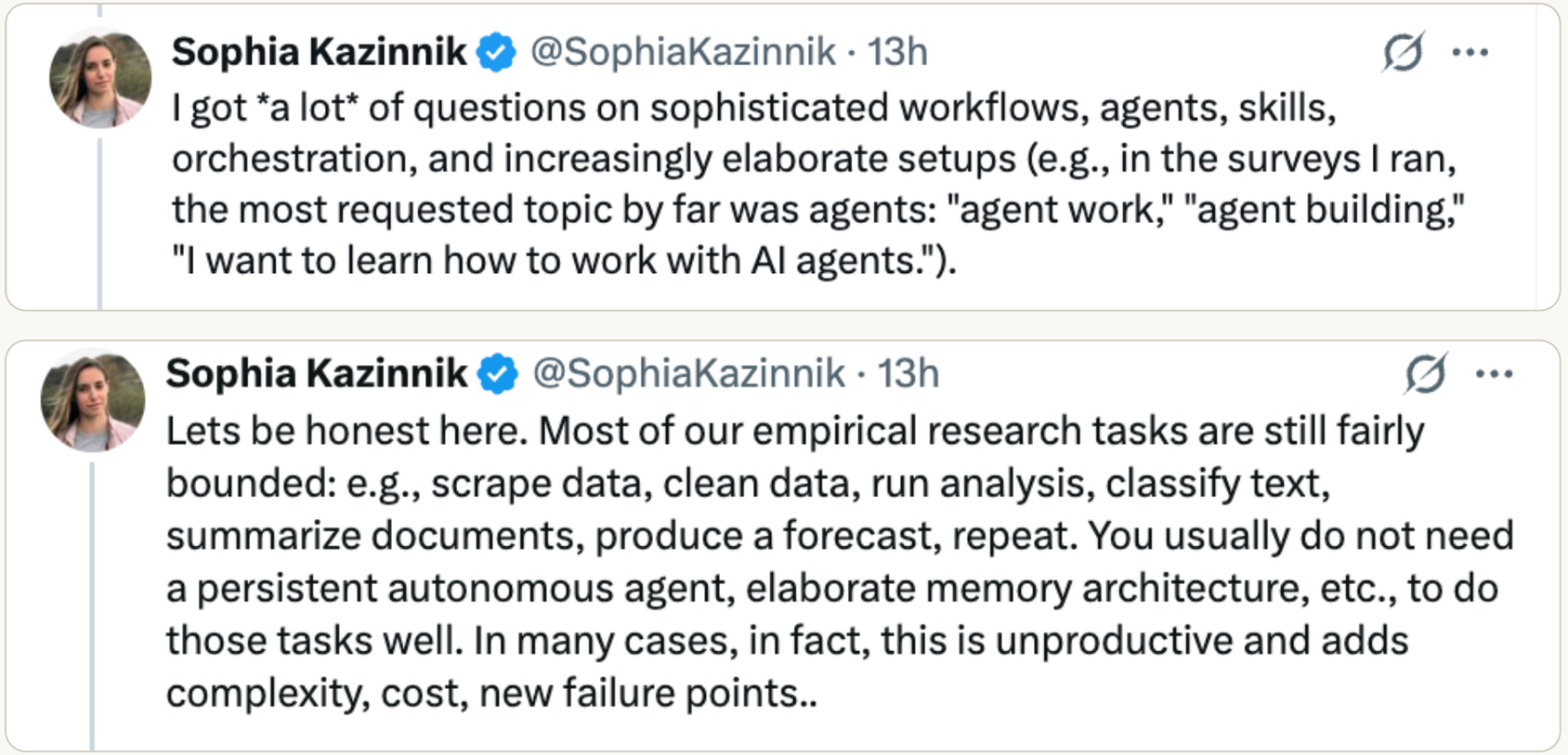
Practical AI for Teaching

Session 2 - Practical workflows

Claes Bäckman
Leibniz Institute for Financial Research SAFE

Most tasks are bounded and do not need elaborate setups



Sophia Kazinnik on X (x.com/SophiaKazinnik). Screenshots as posted.

Before anything: the folder, the panel, the files

1. **VS Code is open with the Claude Code panel in the sidebar.** Five steps if not.
2. **Your project folder is open, not a single file.** File, Open Folder. The folder holds your syllabus, course memo, one week of lecture notes and your thesis rubric (Word or PDF is fine), plus the morning's thesis (`paper.pdf`) from the resources page.
3. **The panel answers.** Type `/` in the panel and a list of commands appears. No exercise needs the skill library; installing it takes two minutes in the break.

`/name` runs a skill, a saved workflow; `@file` points the agent at a file in the folder.

Claude Code needs a paid plan (Pro, Max, Team or Enterprise). Without one, Copilot Chat works for exercises 1, 3, 4 and 5: upload the file and paste the prompt. Exercise 2 needs an agent that reads the folder, so pair up with a neighbour or use GitHub Copilot in VS Code.

VS Code is a file tree, an editor and the agent in one window

Left: the files

The Explorer shows the folder you opened. Click a file to open it. Files the agent writes appear here.

Middle: the editor

The open file. When the agent edits it, the change shows as a diff, added lines and removed lines, and you accept or reject it.

Side: the agent

The Claude Code panel. Type a request, `/` for a skill, `@` for a file. Depending on the permission mode, it asks before it edits or runs a command.

The terminal (`Ctrl+`) is where the agent runs commands. You can watch it, and you rarely need to type in it. Setup in five steps, the extensions, permissions.

Context improves AI tools tremendously, and the goal is to provide the right context.

Five blocks, each the same move: files in, one prompt, one check

1	2	3	4	5
Thesis feedback	**Your own skill**	**AI policy**	**Course development**	**Your own task**
`> read @rubric.md`	`> /thesis-feedback`	`> from @course-memo.md`	`> from notes/`	`> your files`
on the morning's thesis, then verify	from your rubric, for your students	allow, disclose, check	quizzes, updates, lectures	one thing from this morning, on your material

Every block is the same move: put the right files in the folder, type one prompt, and read what comes back as a draft to check, not an answer to accept. The morning explained why. This afternoon you do it. **Break after block 2.**

No Claude Code? Every exercise card says what works in Copilot Chat. **Slides, skills and the morning's thesis:** claesbackman.com/ai_workshop.

You get what you ask for, so try to be creative!

`/explorable-deck` turns notes or a paper into slides in the style of Nicky Case's explorable explanations: one idea per slide, a running example, interactive elements the presenter drives.

What a plain request gives you instead · explorable websites

Examples: a housing-returns paper and a paper on financial advice from LLMs. The skill: github.com/claesbackman/AI-research-feedback, `Skills/explorable-deck`. Adapted from Nityesh Agarwal's skill for explorable websites.

But it comes with some caveats

The source must be good

A muddled source gives a fluent deck about the wrong thing.

You need to practise

Practise more than before: making the slides by hand was how you learned your own lecture, and that part is gone.

1 · Feedback on a thesis

```
> read the thesis and @rubric.md, then assess each criterion
```

Student work stays out of AI tools, so the exercises use the morning's thesis

What may go in

The morning's thesis, which an agent wrote from one prompt. Files the agent reads go to the provider under your subscription's terms.

Students may run something similar on their own drafts, which is difficult to know. You could also provide them with skills.

What stays out

University guidance on generative AI typically says to **never input personal data, student work, exam answers, or other sensitive or confidential information,** even to get feedback.

So also grades, names in course evaluations, and a company's data under a thesis agreement. When in doubt, anonymise, aggregate, or use a made-up example.

Feedback on the morning's thesis, against your own rubric

Exercise 1 · feedback on a thesis

10 min

1. Put the morning's thesis in the folder: `paper.pdf` from the resources page. An agent wrote it, so it is not student work. Use the example `rubric.md`, or put your own rubric next to it.
2. Before you press Enter, write down two guesses: one problem the feedback will catch, and one it will miss.
3. In the Claude Code panel, type: *Read* `@paper.pdf` *and* `@rubric.md`. *For each criterion, give the evidence with section references and the one change that would most improve the thesis. Report, do not rewrite. Save the report as feedback-1.md.*
4. Click *Allow* if it asks to write the file. Then let it run while we talk.

In Copilot Chat: upload paper.pdf and rubric.md, paste the prompt, and save the answer as feedback-1.md yourself.

Debrief at the end of this block: *did it find the problems the morning's questions pointed at? What did it get wrong? What did the adversary find that the report missed?*

The agent that wrote the report will defend it

Ask the agent that wrote the report to check it



system the thesis its own report the conversation so far

Its own answer is in the window. Everything it reads next is read in the light of having written it; it has momentum and a position to defend.

Spawn a fresh subagent to attack it



system the thesis the report, as a file “read this as a hostile examiner”

Same thesis, same report, no history. The report is a file it was handed, not a position it took, and it reads both cold.

Stylised: the widths are not measured. The same report is blue in the first window, where it is the agent’s own output, and brick in the second, where it is a file handed to a reviewer. Both reviewers see the thesis.

You still need to verify the report

Exercise 1b, then debrief

2 + 5 min

1. When `feedback-1.md` is written, start a new conversation and type: *Read* `@feedback-1.md` *and* `@paper.pdf`. *Critique the feedback as a hostile examiner. Report, do not fix.*
2. Read the report against your two guesses. What did it catch that you would not have seen? What did it get wrong?
3. What did the adversary find that the report missed, or overstated?
4. Which criterion would you read first next time, and why?

In Copilot Chat: open a new chat, upload feedback-1.md and paper.pdf, and paste the prompt.

Other tools review papers too: a short list.

You can also produce questions to ask for a defence

Exercise 1c · defence questions

5 min

1. In a new conversation, type the prompt from this morning:

> *Read the thesis in this folder. Write five questions for the opponent to ask at the defence, each answerable in under a minute without the text: one on the research question, one on the data, one on a specific table or figure, one on a choice the student made, and one on what the result does not show. For each, say what a good answer contains.*

2. Check that every question points at something that is really in the thesis. Delete the generic ones.

In Copilot Chat: upload paper.pdf and paste the prompt.

Debrief: *which question would you actually ask? Which one tests nothing?*

2 · Writing your own skill

```
> make this a skill called thesis-feedback
```

Turn your rubric into a skill

Exercise 2 · `/thesis-feedback` **15 min**

1. Type: *Make a skill called thesis-feedback in .claude/skills that does what my exercise 1 prompt did: read the thesis and rubric.md, give the evidence per criterion with section references, report without rewriting, and save the report as feedback.md.*
2. Open the `SKILL.md` it wrote and read it. You can change it yourself!
3. Start a new conversation, type `/thesis-feedback`, and run it on the same thesis.
4. Compare `feedback.md` with `feedback-1.md`. Edit `SKILL.md` once more for what it still gets wrong.

No agent? GitHub Copilot in VS Code reads `.claude/skills` too. In Copilot Chat, ask for the SKILL.md text, save it yourself, and paste it as the first message of a new chat with the thesis.

Debrief: *which instruction did you change first? Would you give this skill to your students?*

3 · Writing and the AI policy

```
> from @course-memo.md, draft the AI policy for the thesis course
```

Tell students to use AI to edit their draft, not to edit the AI's draft

Writing is thinking

Remove too much of the writing friction and it gets harder to find out what a thesis is about. The first draft is where the argument is discovered.

Readers discount prose they think is generated

Once a reader decides a paragraph was generated, by instinct or with a detector, they discount the idea in it. That is a cost to the idea, not to the prose.

For your own prose: a voice file keeps AI edits sounding like you.

Write the AI policy: what is allowed, what is disclosed, what is checked

Exercise 3 · an AI policy

5 min

1. Put your course memo in the folder: Word, PDF or markdown. No memo? Use the syllabus.
2. Type: *From the course memo and the thesis schedule, draft an AI policy for the course: what students may use AI for, what they must disclose and how, and which parts the defence will test without the text. Write in plain language, for students.*
3. Then read, verify, edit and tell the AI what you want changed.

In Copilot Chat: upload the course memo and paste the prompt.

Debrief: *what did it allow that you would not?*

Allowed. Editing, debugging, explaining a method. Or not: you decide, per assignment.

Disclosed. One sentence, the prompts, or the full log.

Checked. What the student explains without the text.

Bryan's rule 8: the AI policy goes in the syllabus, per assignment (Bryan 2026).

4 · Developing and updating a course

```
> from @notes/week3.md, write 15 multiple-choice questions at three levels
```

Build a weekly quiz bank from your notes

Exercise 4 · a quiz bank

10 min

1. Put one week's lecture notes in a `notes/` folder: markdown, LaTeX or PDF all work. A `.pptx`? Ask the agent to convert it first.
2. Guess first: how many of the 15 answer keys will be wrong? Then type the prompt:

> *From the notes in* `notes/`*, write 15 multiple-choice questions at three levels, with distractors built from common misconceptions and a one-sentence explanation each.*

3. Read every question. **Verify every answer key yourself.**

In Copilot Chat: upload one week's notes and paste the prompt.

Updating a course: put the outcomes and the guidelines in the folder first

Exercise 5 · optional · a course update

10 min

1. Put the syllabus, the course memo and the list of lectures in the folder. If your university or department has guidance on sustainability or gender equality in teaching, add it.
2. Type the prompt:

> *From the syllabus, the course memo and the guidance in this folder, propose where a sustainability or gender perspective fits in this course: one concrete example per week, each tied to an existing learning outcome, with a reading or dataset I can check. Do not change any files.*

3. Check that every reading exists. Or start from last year's course analysis: *list the three changes students asked for most, and propose how to make each.*

In Copilot Chat: upload the syllabus and the course memo, and paste the prompt.

5 · Your own task

```
> read the files in my-course/ and …
```

Pick one thing from this morning and do it on your own material

Update a lecture

Slides with 2015 examples get current ones. The structure stays.

Write a rubric

From the course's learning outcomes. Then test it on the morning's thesis.

Build a data exercise

From a public source: SCB, ECB, Eurostat, World Bank. Plan first.

Next year's course memo

Last year's memo, the new template and this year's dates. You check.

A skill for one criterion

The criterion students miss most often, as a skill they can run on their draft.

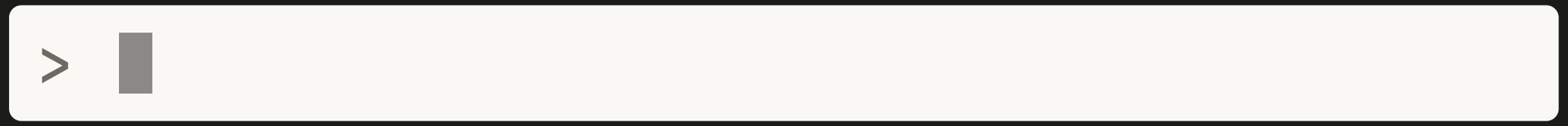
Problem sets with solutions

At three levels, with a rubric. Check every key yourself.

The same move as every block: the files in the folder, one prompt, one check. Twenty minutes, and I come round.

Your turn

Thank you!



Questions now or later: claes.backman@gmail.com. Slides, skills, guides: claesbackman.com/ai_workshop

Appendix

```
> the rooms, for questions
```

Five steps get VS Code and Claude Code working in under five minutes

1. Open VS Code and the Extensions panel (`Ctrl+Shift+X`, `Cmd+Shift+X` on a Mac).
2. Search "Claude Code" and install *Claude Code for VS Code* (extension ID `anthropic.claude-code`).
3. Restart VS Code. Claude Code opens in the sidebar.
4. Sign in with your Claude account (Pro, Max, Team and Enterprise all include Claude Code), or an API key.
5. Open your **full project folder** (File, Open Folder), never a single file.

What an IDE is. Editor, file explorer, terminal and extensions in one window; think of it as Word or RStudio for code, LaTeX, markdown and plain text. VS Code is the dominant free one.

Then: the extensions, every session, installing the skills. Back to the check-in.

Installing the skills is a copy into one hidden folder

1. Download the zip from claesbackman.com/ai_workshop, or paste the one-line command from that page into the VS Code terminal (`Ctrl+`). Source and updates: github.com/claesbackman/AI-research-feedback. `/voice-extractor` is Mihail Velikov's and is linked from the same page.
2. Unzip, then copy each skill folder (`review-paper`, `review-paper-light`, `voice-extractor`, ...) into `~/.claude/skills/`, a hidden folder under your user folder. Windows: `C:\Users\you\.claude\skills`; to see hidden folders, View, Show, Hidden items in File Explorer.
3. A skill in `~/.claude/skills/` works in every project; one in `project/.claude/skills/` only in that project.
4. Start a new Claude Code conversation. Type `/` and the skills appear in the list.

Back to the check-in or exercise 2.

A handful of extensions cover Stata, Python, LaTeX and data

STACK	EXTENSION ID	WHAT IT DOES
Stata	`tmonk.stata-workbench`	Run Stata from the editor
Python	`ms-python.python`	Syntax, IntelliSense, run button
LaTeX	`james-yu.latex-workshop`	Compile on save, side-by-side PDF preview, inline errors
Data	`mechatroner.rainbow-csv`	Colour-coded CSV columns; SQL-like filtering
	`tomoki1207.pdf`	Renders PDFs inline: read codebooks next to your code

Install Python even if you do not write it. The agent uses Python as glue for many different tasks.

Back to setup.

Every session: open the folder, set context, then prompt

1. Open VS Code and the project folder (File, Open Folder).
2. Open the Claude Code panel in the sidebar.
3. Confirm Claude is pointed at the full project folder, not a single file.
4. Send a first orienting prompt.

> *Read the files in this folder and tell me what this project is about.*

Expected behaviour. Claude reads the folder, summarises the structure, and asks what you want to do next.

Then: permissions, a session is a conversation. Back to setup.

Permissions follow a tiered model, and auto mode is now the default

Plan mode	`reads only`	Reads and proposes; cannot write.
Manual	`asks first`	Asks before each edit or command.
Edit automatically	`edits files`	Edits your selected text or the whole file without asking.
Auto mode	`the default now`	Approves actions that pass a safety check and pauses for anything risky.

Claude Code is meant to stay inside the folder you opened; do not rely on it for a folder that holds anything sensitive.
`/permissions` shows the current rules; allow- or block-list specific commands (`Rscript`, `pdflatex`, `rm`) in `settings.json`.
Manual mode for the first week is a reasonable choice.

Back to every session.

A session is a conversation, not a query

1. **Prompt:** specific, with the context it needs.
2. **Response:** a draft, not an answer.
3. **You react:** accept, correct, redirect. The next turn builds on your corrections; ten to twenty turns is normal.

Continue the conversation. “Shorter, and use a Swedish example.” “You misread Table 2; look again.” “Now the same for Section 3.”

Off track? Kill it and restart with a better first prompt

Signs: it repeats a mistake, argues in circles, forgets an earlier instruction, or three turns go on fixing one thing.

Start fresh and put what you learned into the first prompt, or into `CLAUDE.md` so you never repeat it. Signs of drift.

Back to every session.

Signs of session drift

1. Claude ignores `CLAUDE.md`
2. Repeats mistakes you already corrected
3. Invents files that do not exist
4. Gives circular answers
5. Forgets the last three turns

Fix. Start a fresh conversation (`/clear`), or close and reopen. Re-state the task. Point at the relevant file again.

Thirty turns, or the task has shifted: start fresh.

Back to a session is a conversation.

A few files hold your durable project context

FILE	USE IT FOR
`project/CLAUDE.md`	Project instructions: dataset structure, sample restrictions, paper title, coauthor conventions, which script produces which output
`~/.claude/CLAUDE.md`	Personal preferences across every project: writing style, default tone, citation conventions. Project instructions load later and usually carry more weight
`voice.md`	Writing samples: paragraphs, abstracts and tone notes Claude should match when editing prose
`.claude/skills/…`	Reusable workflows: named tasks you run again without rewriting the prompt

Keep secrets and raw confidential data **out of all of them**: they are read into context every session and sent to the model provider.

Then: bootstrap with /init, the starter template. Back to the voice file.

/init writes a first CLAUDE.md in two minutes

The built-in command. Type `/init` in the Claude Code panel. Or spell it out:

> *Read this project folder and draft a* `CLAUDE.md` *file with the project structure, conventions, and instructions you should follow. Do not edit anything else yet.*

What Claude does. Reads the folder, infers languages, structure and conventions, and writes a draft.

Then you edit it. Add tone preferences, things to avoid, and the sample restrictions it could not infer. The draft is the starting point, not the final word.

Back to the files or the voice exercise.

CLAUDE.md starter template

```
# About me
I teach [course] and supervise theses in [field].

# How I want you to work with me
- Ask clarifying questions before long output.
- Critical, skeptical tone in feedback. Do not flatter.
- When editing, preserve my voice (see voice.md).
  No generic LLM phrasing.
- No bullet lists or passive voice in formal writing.
- Cite the line or section you are commenting on.

# Things to avoid
- Do not fabricate citations.
- Do not over-claim causality.
- No emoji or markdown decorations in formal docs.
```

Always loaded, every session: the things you would otherwise repeat in every prompt. Keep it to a page; long workflows go in skills.

Back to exercise 2 or the files.

A voice file puts your prose in the window

> *When editing my prose, match the voice in* `voice.md`*. Do not use em-dashes, bullet lists, or passive constructions unless the original does. Do not use "it's not x, it's y" constructions.*

What goes in. Five to ten thousand words of your own prose: introductions, abstracts, paragraphs you are proud of. The skill picks them out of the PDFs you give it.

What stays out. Coauthored passages that are not yours. Anything AI-edited.

Where it goes. Next to `CLAUDE.md`. Point at it from there, so every session loads your voice, or with `@voice.md` only when editing, to keep the permanent window small. The files that hold durable context.

Analyse your own voice

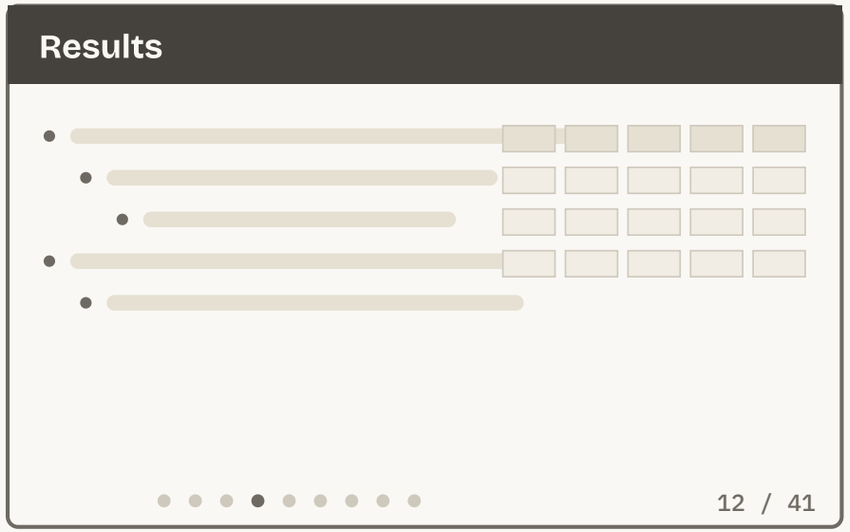
Optional · `/voice-extractor` 15 min

1. Inside the project folder, make a folder `sample-papers` with three or four PDFs of your own writing. Coauthored? Pick the papers where the introduction is yours.
2. Type `/voice-extractor sample-papers`. No skill? Type *Read the PDFs in sample-papers and write voice.md: how I build paragraphs, sentence length, words I use and avoid, with quoted examples.*
3. Read `voice.md` and delete or edit anything that does not sound like you.
4. Add one line to `CLAUDE.md`: "When editing my prose, match the voice in voice.md." No `CLAUDE.md` yet? Type `/init` first.

Debrief: *which sentence in* `voice.md` *did you delete first? Did it name a habit you did not know you had?*

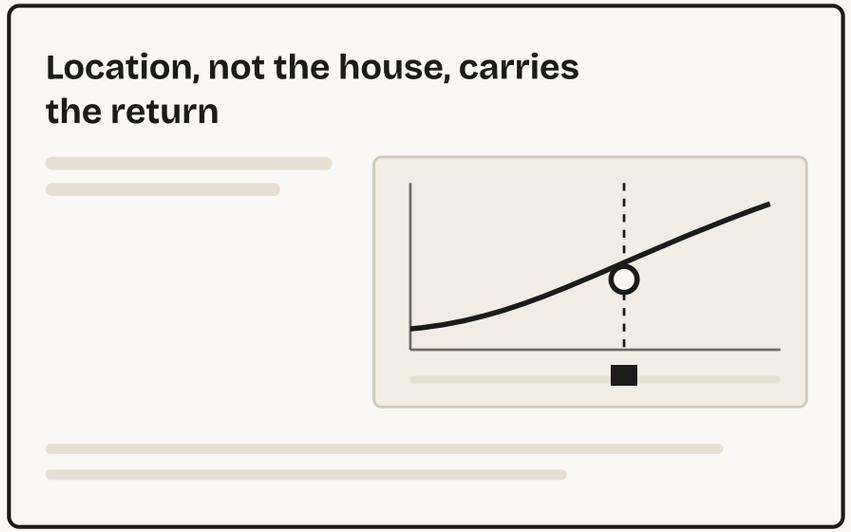
Back to the voice file or writing.

Ask for academic slides and you get a stochastic Beamer parrot



the parrot

a working deck in twenty minutes
that looks like every other deck online



an explorable deck

a claim for a title, one idea, a live stage
the new thing, not the old thing faster

Stylised: two slides drawn from memory, not screenshots. Left, what “make a Beamer deck from the paper” produces. Right, one slide from an explorable deck of a housing-returns paper, with its handle. Both take about twenty minutes to draft.

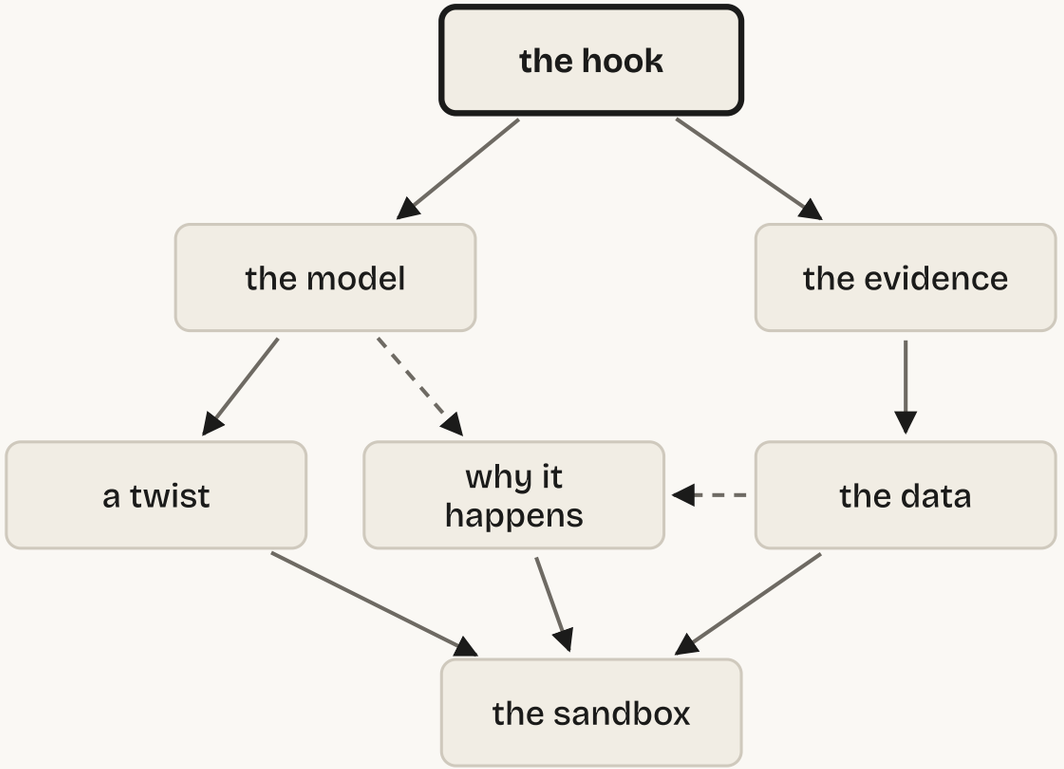
Back to explorable decks.

Other fields have worked out how to explain their work

`/explorable-explanation` builds a website as a non-linear tree of short HTML decks, one idea per slide, with forks placed where the reader becomes curious.

How it works

1. Reads the source material, then writes `plan.md` like a script: the hook, what the reader sees and does, and where the branches go
2. Shows you the first batch before it builds the rest
3. Screenshots every slide and reworks the ones that break the craft rules
4. Sends adversarial subagents to judge writing, interface, visuals and teaching
5. A “creative director” subagent gives it a visual identity; an impatient-reader subagent flags every slide where the flow breaks



Each box is a short deck. Dashed arrows are forks the reader chooses from inside a slide.

Back to explorable decks.

The skill library

SKILL	USE
`/review-paper`	Pre-submission referee report, 8 reviewers
`/review-paper-light`	Fast 2-reviewer version, about 1 minute
`/review-paper-code`	Paper and code alignment review
`/review-grant`	Pre-submission grant review, 6 reviewers
`/review-pap`	Pre-analysis-plan review
`/audit-analysis`	Adversarial audit of changed analysis code
`/explain-diff`	Explain a code change as an HTML page with a quiz
`/paper-version`	Paper to policy brief, 1-page or 5-page summary
`/pdf-to-markdown`	Convert PDFs to readable markdown
`/prose-revision`	Revise idea flow before wording
`/voice-extractor`	Write `voice.md` from PDFs of your own prose (Mihail Velikov)

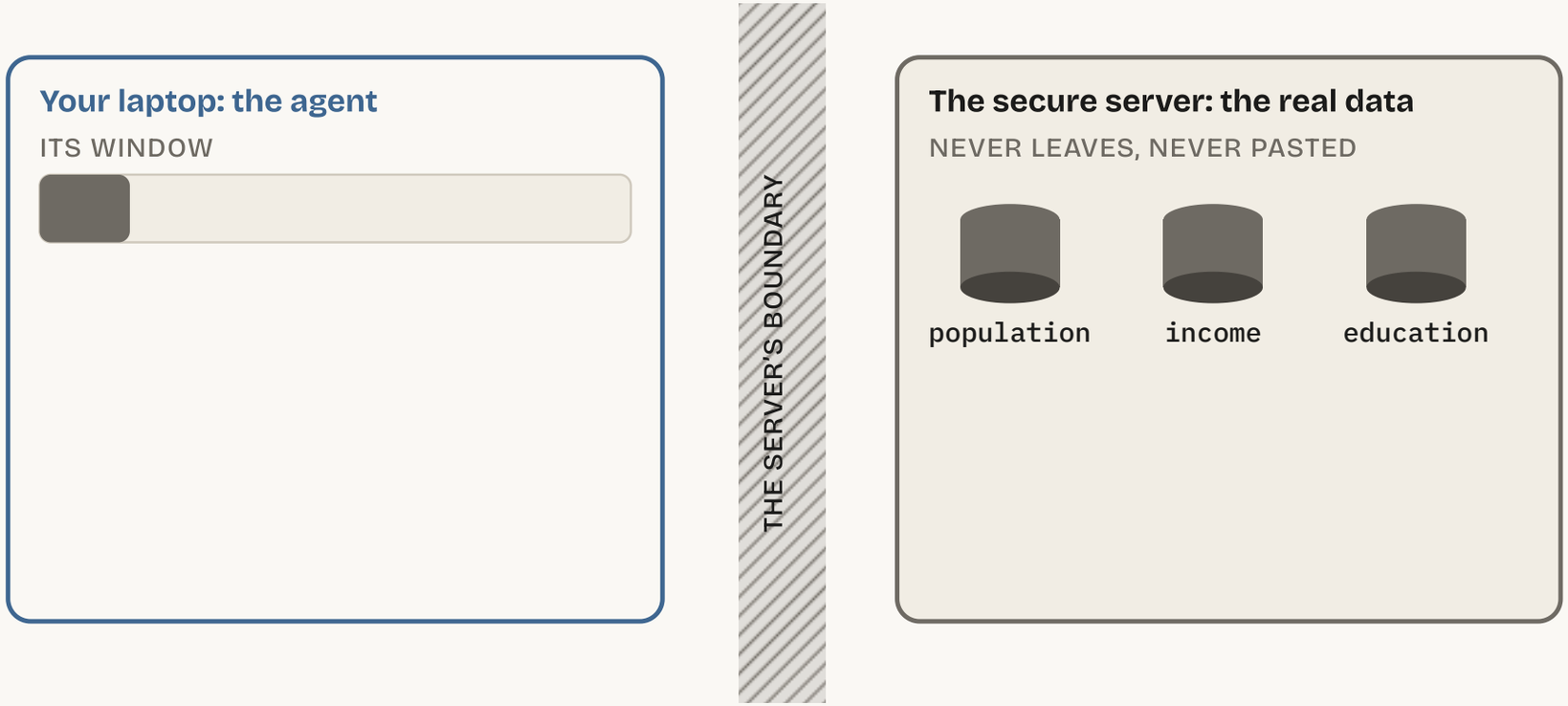
Mine are at github.com/claesbackman/AI-research-feedback (MIT); all are linked from claesbackman.com/ai_workshop. Fork them, modify them, make them yours. Back to exercise 2.

File formats: text is free, binaries cost a conversion

FORMAT	HANDLING
`.md`, `.tex`, `.txt`, `.bib`	Native, no conversion
`.py`, `.R`, `.do`, `.ipynb`	Native: read, edit, run
`.csv`	Native for a few thousand rows; larger files it inspects through code, a few rows at a time
`.docx`, `.pptx`	Convert: `pandoc file.docx -o file.md`, or ask the agent to
`.xlsx`	Export to CSV first
`.dta`, `.rds`	Read through code, not as text
PDF, born-digital and short	Native; expect table mangling
PDF, long, scanned or with equations	Convert to markdown first: `/pdf-to-markdown`

Back to exercise 1 or the two files.

The agent never sees the data, but it sees everything that explains the data



Step Reset Press **step**. Six steps; the wall is never crossed by data.

Stylised. Brick: what is in the agent's window. Grey: the registers the agent never sees. The dashed arrow is you carrying the do-file. The struck-out bubble: pasting the log into a chat window is the agent reading it. What crosses back is what you type: an error code or a request, never a result.

On to the two files.

Two files to keep local: a codebook in markdown and a simulated dataset

1 · Codebook in markdown

Convert the PDF once with `/pdf-to-markdown`. Variable names, labels, value ranges, missing-value codes.

The agent reads it before it writes a `recode`, a `label define` or a `merge`.

2 · Simulated dataset

Same variable names, types and ranges; not the real values. Some replication packages ship one.

Or ask the agent to generate it from the codebook: one prompt, fifty thousand rows.

Back to the workflow, on to the caveat.

But synthetic data tests that the code runs, not that it is right

Tests that the code runs

Variable names, merge syntax, reshapes, type handling, the shape of the panel. Everything that would fail on the schema fails on your laptop, before you enter the secure environment.

Does not test that it is right

Correlations, magnitudes, selection, missingness, sign conventions, duplicate keys and gaps that only the real values produce. The fake data contains what someone thought to put in it.

Back to the two files.

GitHub or Dropbox?

	GITHUB	DROPBOX
Sync	Manual push and pull, in GitHub Desktop, no terminal	Automatic, within seconds
History	Full history, diffs, branches	Dropbox revisions only
Best for	Versioned drafts; the review skills	Live coauthoring in Overleaf
Conflicts	A merge you resolve	“Conflicted copy” files

On to Overleaf.

You can link Overleaf to your own computer and work in that folder

Why bother. Overleaf has no Claude Code, no Codex, no Copilot, no proper diff view, no terminal. The fix: write in VS Code and let Overleaf follow what you produce.

Requirement. Both routes need Overleaf Premium; many universities provide it through the institutional login.

Two routes. GitHub via GitHub Desktop for a history, click to commit, no terminal. Dropbox for live coauthoring, no commit step. Click by click.

claesbackman.com/overleaf-vscode-guide; Overleaf documentation. Back to GitHub or Dropbox.

The two Overleaf routes, click by click

Setup A · GitHub via GitHub Desktop

- Overleaf: Menu, Integrations, GitHub, **Export Project to GitHub**
- GitHub Desktop: File, Clone repository; pick the repo and a local folder
- Open that folder in VS Code and edit
- GitHub Desktop: one-line summary, **Commit to main**, **Push origin**
- Overleaf: Menu, GitHub, **Pull GitHub changes into Overleaf**
- Coauthor edited in Overleaf? **Push Overleaf changes to GitHub**, then pull in GitHub Desktop

Accept the TeX `.gitignore` template: no `.pdf` or `.aux` in the history.

Setup B · Dropbox

- Install Dropbox and let the initial sync finish
- Overleaf: Menu, Dropbox Sync, **Link**. Overleaf creates `Apps/Overleaf/` with one subfolder per project
- Open the project folder in VS Code and edit as normal; every save reaches Overleaf within seconds, and Overleaf edits flow back
- Compile in Overleaf as usual, or locally with `latexmk -pdf`

No commit step. Good for active coauthoring, less good if you want a history.

Back to Overleaf.

Other tools generate referee reports too

TOOL	WHAT IT IS
Refine.ink	Referee-style report on a paper PDF; paid
coarse.ink	Lightweight version of the same idea; close to free
Referee 2	Referee-report service; paid
Stanford Agentic Reviewer	Open agentic reviewer; free

A skill that lives next to your draft knows your conventions and lets you edit what it checks. The services do not.

Back to the debrief.

/prose-revision fixes the order of ideas before the choice of words

What comes back

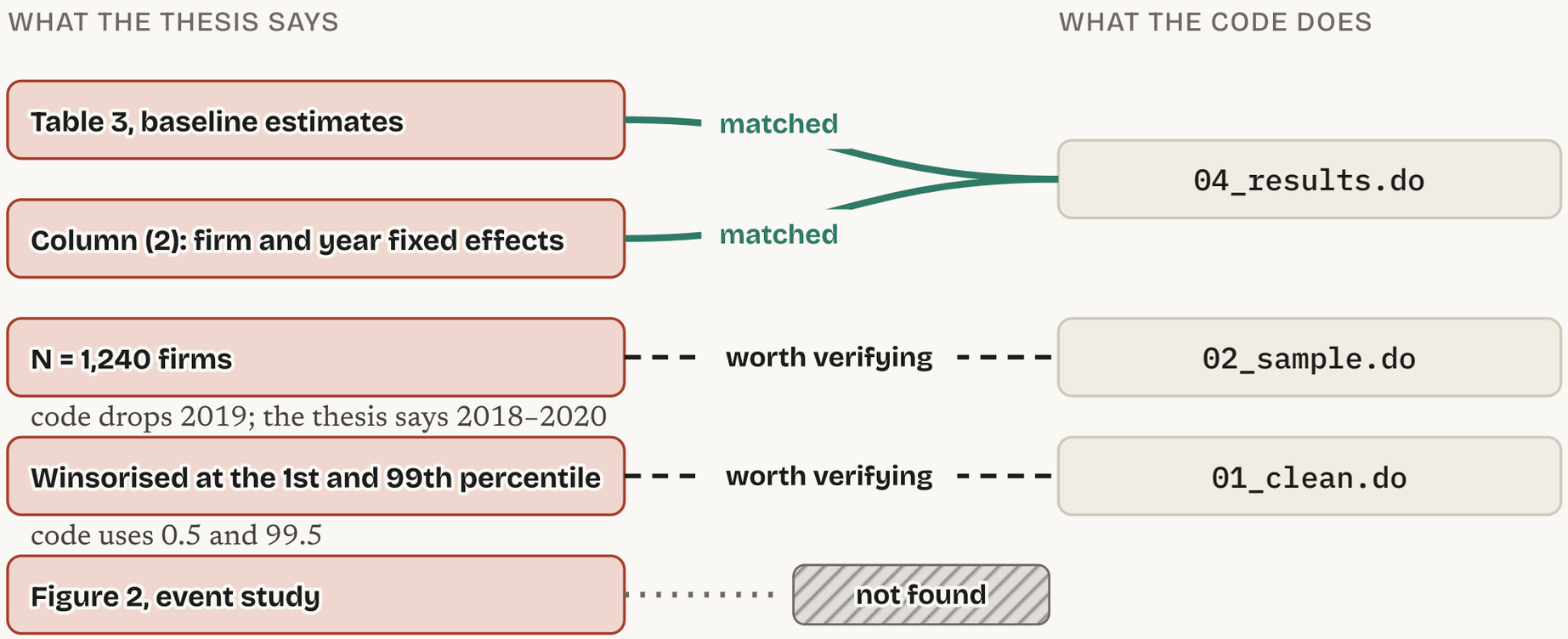
A diagnosis of the idea flow first. Then two or three rewrites that differ in how far they restructure, and a short note on the trade-off between them. Claims, citations and notation come through unchanged.

A skill in the library, after a method described by alz_zyd on X (x.com/alz_zyd_).

Where it stops

It will not add or drop an idea without saying so, and it is not a proofreader. Revision first, the typo pass last.

/review-paper-code maps a thesis's claims to the scripts that produce them



A stylised example of the thesis–code map the skill writes. The five rows are invented for the slide; the three statuses are the report's. Brick: claims read from the thesis. Teal: the code does what the thesis says. Dashed: the two disagree, and that row is read first. Hatched: no script produces it.

Stylised example. The skill needs the thesis in LaTeX and the code in Stata, R or Python.

References

Bryan, Kevin A. 2026. *Eight Rules for Teaching in AI World*. Https://kevinbryanecon.com/teachwithai.html.